

Методические указания к лабораторной работе на тему «Алгоритмы фрактальной графики»

Системы итерируемых функций

Метод "Систем Итерируемых Функций" (Iterated Functions System – IFS) появился в середине 80-х годов как простое средство получения фрактальных структур.

IFS представляет собой систему функций из некоторого фиксированного класса функций, отображающих одно многомерное множество на другое. Наиболее простая IFS состоит из аффинных преобразований плоскости:

$$\begin{aligned}X' &= A*X + B*Y + C, \\Y' &= D*X + E*Y + F.\end{aligned}$$

В 1988 году известные американские специалисты в теории динамических систем и эргодической теории Барнсли и Слоан предложили некоторые идеи, основанные на соображениях теории динамических систем, для сжатия и хранения графической информации. Они назвали свой метод методом фрактального сжатия информации. Происхождение названия связано с тем, что геометрические образы, возникающие в этом методе, обычно имеют фрактальную природу в смысле Мандельброта.

На основании этих идей Барнсли и Слоан создали алгоритм, который, по их утверждению, позволит сжимать информацию в 500-1000 раз. Коротко метод можно описать следующим образом. Изображение кодируется несколькими простыми преобразованиями (в нашем случае аффинными), т.е. коэффициентами этих преобразований (в нашем случае A,B,C,D,E,F).

Например, закодировав какое-то изображение двумя аффинными преобразованиями, мы однозначно определяем его с помощью 12-ти коэффициентов. Если теперь задаться какой-либо начальной точкой (например, $X=0$ $Y=0$) и запустить итерационный процесс, то мы после первой итерации получим две точки, после второй – четыре, после третьей – восемь и т.д. Через несколько десятков итераций совокупность полученных точек будет описывать закодированное изображение. Но проблема состоит в том, что очень трудно найти коэффициенты IFS, которая кодировала бы произвольное изображение.

Для построения IFS применяют кроме аффинных и другие классы простых геометрических преобразований, которые задаются небольшим числом параметров. Например, проективные:

$$\begin{aligned}X' &= (A1*X + B1*Y + C1) / (D1*X + E1*Y + F1), \\Y' &= (A2*X + B2*Y + C2) / (D2*X + E2*Y + F2),\end{aligned}$$

или квадратичные:

$$\begin{aligned}X' &= A1*X*X + B1*X*Y + C1*Y*Y + D1*X + E1*Y + F1, \\Y' &= A2*X*X + B2*X*Y + C2*Y*Y + D2*X + E2*Y + F2,\end{aligned}$$

преобразования на плоскости.

В качестве примера использования IFS для построения фрактальных структур, рассмотрим кривую Коха и "дракона" Хартера-Хейтуэя. Выделим в этих структурах подобные части и, для каждой из них вычислим

коэффициенты аффинного преобразования. В аффинный коллаж будет включено столько аффинных преобразований, сколько существует частей подобных целому изображению.

Построим IFS для "дракона" Хартера-Хейтуэя. Для этого расположим первое поколение этого фрактала на сетке координат дисплея 640 x 350 (рис. 1). Обозначим точки получившейся ломаной **A**, **B**, **C**. По правилам построения у этого фрактала две части, подобные целому – на рис. 1 это ломаные **ADB** и **BEC**. Зная координаты концов этих отрезков, можно вычислить коэффициенты двух аффинных преобразований, переводящих ломаную **ABC** в **ADB** и **BEC**:

$$X' = -0.5 * X - 0.5 * Y + 490,$$

$$Y' = 0.5 * X - 0.5 * Y + 120,$$

и

$$X' = 0.5 * X - 0.5 * Y + 340,$$

$$Y' = 0.5 * X + 0.5 * Y - 110.$$

Задавшись начальной стартовой точкой (например $X=0$ $Y=0$) и итерационно действуя на нее этой IFS, после десятой итерации на экране получим фрактальную структуру, изображенную на рис. 2, которая представляет собой "дракон" Хартера-Хейтуэя. Его кодом (сжатым описанием) является набор коэффициентов двух аффинных преобразований.

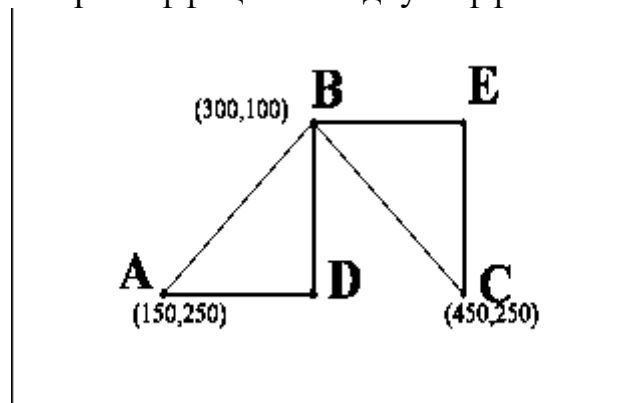


Рис. 1. Заготовка для построения IFS "дракона" Хартера-Хейтуэя

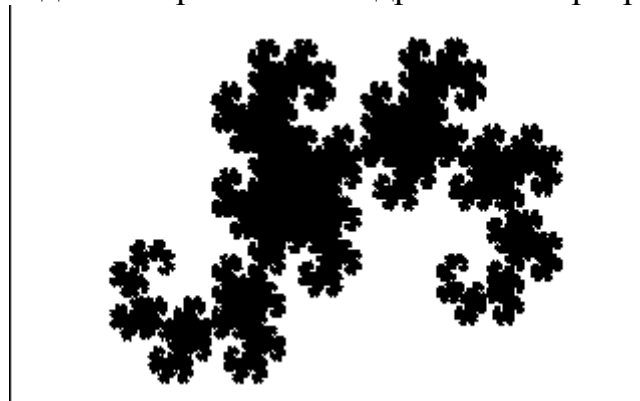


Рис. 2. "Дракон" Хартера-Хейтуэя, построенный с помощью IFS в прямоугольнике 640x350

Аналогично можно построить IFS для кривой Кох. Нетрудно видеть, что эта кривая имеет четыре части, подобные целой кривой (см. рис. 3). Для

нахождения IFS опять расположим первое поколение этого фрактала на сетке координат дисплея 640 x 350 (рис. 3).

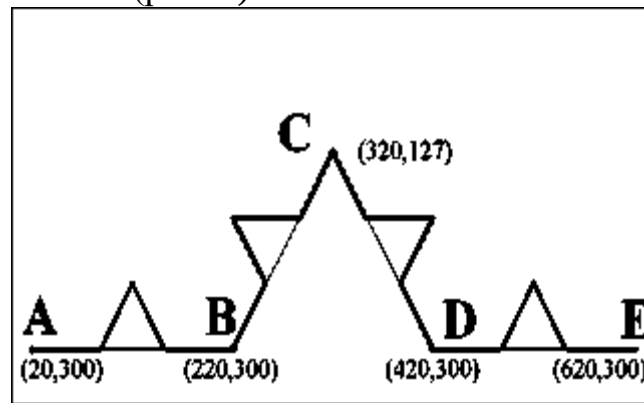


Рис. 3. Заготовка для построения IFS кривой Кох

Для ее построения требуется набор аффинных преобразований, состоящий из четырех преобразований:

$$X' = 0.333 * X + 13.333$$

$$Y' = 0.333 * Y + 200$$

$$X' = 0.333 * X + 413.333$$

$$Y' = 0.333 * Y + 200$$

$$X' = 0.167 * X + 0.289 * Y + 130$$

$$Y' = -0.289 * X + 0.167 * Y + 256$$

$$X' = 0.167 * X - 0.289 * Y + 403$$

$$Y' = 0.289 * X + 0.167 * Y + 71.$$

Результат применения этого аффинного коллажа после десятой итерации можно увидеть на рис. 4.

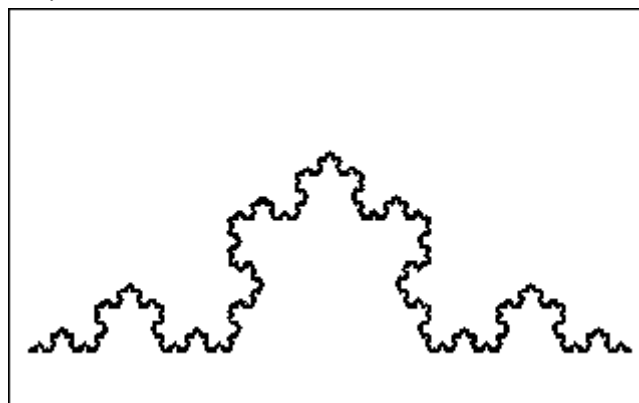


Рис. 4. Кривая Кох, построенная с помощью IFS в прямоугольнике 640x350

Использование IFS для сжатия обычных изображений (например фотографий) основано на выявлении локального самоподобия, в отличие от фракталов, где наблюдается глобальное самоподобие и нахождение IFS не слишком сложно (мы сами только-что в этом убедились). По алгоритму Барнсли происходит выделение в изображении пар областей, меньшая из которых подобна большей, и сохранение нескольких коэффициентов, кодирующих преобразование, переводящее большую область в меньшую. Требуется, чтобы множество "меньших" областей покрывало все

изображение. При этом в файл, кодирующий изображения будут записаны не только коэффициенты, характеризующие найденные преобразования, но и местоположение и линейные размеры "больших" областей, которые вместе с коэффициентами будут описывать локальное самоподобие кодируемого изображения. Восстанавливающий алгоритм, в этом случае, должен применять каждое преобразование не ко всему множеству точек, получившихся на предыдущем шаге алгоритма, а к некоторому их подмножеству, принадлежащему области, соответствующей применяемому преобразованию.

Пример построения папоротника Барнсли на С#.

Папоротник Барнсли - фрактал, названный в честь Майкла Барнсли, британского математика, который первым описал его в своей книге "Фракталы Повсюду". Является одним из основных примеров "самоподобных" множеств, т.е. представляет собой математически генерируемый "шаблон", воспроизводимый при любом увеличении или уменьшении количества итераций.

Построение

Папоротник Барнсли строится при помощи 4-х аффинных преобразований вида:

$$f(x, y) = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} e \\ f \end{bmatrix}$$

Барнсли представил свой IFS-код папоротника (Iterated function system - "Система повторяющихся функций") в виде матрицы значений:

w	a	b	c	d	e	f	p
f1	0	0	0	0.16	0	0	0.01
f2	0.85	0.04	-0.04	0.85	0	1.6	0.85
f3	0.2	-0.26	0.23	0.22	0	1.6	0.07
f4	-0.15	0.28	0.26	0.24	0	0.44	0.07

где столбцы a-f - коэффициенты уравнения, а p - коэффициент вероятности, x и y - координаты.

Данная таблица отвечает следующим преобразованиям:

$$f(x, y) = \begin{bmatrix} 0.00 & 0.00 \\ 0.00 & 0.16 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

$$f(x, y) = \begin{bmatrix} 0.85 & 0.04 \\ -0.04 & 0.85 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} 0.00 \\ 1.60 \end{bmatrix}$$

$$f(x, y) = \begin{bmatrix} 0.20 & -0.26 \\ 0.23 & 0.22 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} 0.00 \\ 1.60 \end{bmatrix}$$

$$f(x, y) = \begin{bmatrix} -0.15 & 0.28 \\ 0.26 & 0.24 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} 0.00 \\ 0.44 \end{bmatrix}$$

Фрактал папоротник Барнсли строится при помощи библиотеки System.Drawing.

Создается проект Windows Form, на форму добавляется PictureBox (FernPictureBox, размер - 800x640)

Текст программы:

```
using System;
using System.Drawing;
using System.Windows.Forms;

namespace Fern
{
    public partial class Form1 : Form
    {
        // Задаем диапазон значений для точек
        private const float MinX = -6;
        private const float MaxX = 6;
        private const float MinY = 0.1f;
        private const float MaxY = 10;
        // Количество точек для отрисовки
        private const int PointNumber = 200000;
        // Массив коэффициентов вероятностей
        private float[] _probability = new float[4]
        {
            0.01f,
            0.06f,
            0.08f,
            0.85f
        };
        // Матрица коэффициентов
        private float[,] _funcCoef = new float[4, 6]
        {
            //a      b      c      d      e  f
            {0,      0,      0,      0.16f, 0, 0}, // 1 функция
            {-0.15f, 0.28f, 0.26f, 0.24f, 0, 0.44f}, // 2 функция
            {0.2f,   -0.26f, 0.23f, 0.22f, 0, 1.6f}, // 3 функция
            {0.85f,  0.04f, -0.04f, 0.85f, 0, 1.6f} // 4 функция
        };
        // коэффициент масштабируемости высоты и ширины
        // изображения фрактала для высоты и ширины нашей формы
        private int _width;
        private int _height;
        // Bitmap для папоротника
        private Bitmap _fern;
        // используем для отрисовки на PictureBox
        private Graphics _graph;

        public Form1()
        {
            InitializeComponent();

            // вычисляем коэффициент
```

```

        _width = (int) (FernPictureBox.Width / (MaxX - MinX));
        _height = (int) (FernPictureBox.Height / (MaxY - MinY));
        // создаем Bitmap для папоротника
        _fern = new Bitmap(FernPictureBox.Width, FernPictureBox.Height);
        // создаем новый объект Graphics из указанного Bitmap
        _graph = Graphics.FromImage(_fern);
        // устанавливаем фон
        _graph.Clear(Color.Black);

        DrawFern();
    }

    private void DrawFern()
    {
        Random rnd = new Random();
        // будем начинать рисовать с точки (0, 0)
        float xtemp = 0, ytemp = 0;
        // переменная хранения номера функции для вычисления следующей
        точки
        int numF = 0;

        for (int i = 1; i <= PointNumber; i++)
        {
            // случайное число от 0 до 1
            var num = rnd.NextDouble();
            // проверяем какой функцией воспользуемся для вычисления
            следующей точки
            for (int j = 0; j <= 3; j++)
            {
                // если случайное число оказалось меньше или равно
                // заданного коэффициента вероятности,
                // задаем номер функции
                num = num - _probability[j];
                if (num <= 0)
                {
                    numF = j;
                    break;
                }
            }

            // вычисляем координаты
            var x = _funcCoef[numF, 0] * xtemp + _funcCoef[numF, 1] *
ytemp + _funcCoef[numF, 4];
            var y = _funcCoef[numF, 2] * xtemp + _funcCoef[numF, 3] *
ytemp + _funcCoef[numF, 5];

            // сохраняем значения для следующей итерации
            xtemp = x;
            ytemp = y;
            // вычисляем значение пикселя
            x = (int) (xtemp * _width + FernPictureBox.Width / 2);
            y = (int) (ytemp * _height);
            // устанавливаем пиксель в Bitmap
            _fern.SetPixel((int)x, (int)y, Color.LawnGreen);
        }
        // Отображаем результат
        FernPictureBox.BackgroundImage = _fern;
    }
}

```



Задание

В соответствии со своим вариантом задания реализовать фрактал, заданный системой коэффициентов.

Вариант 1.

```
Crystal {
0.696970 -0.481061 -0.393939 -0.662879 2.147003 10.310288 0.747826
0.090909 -0.443182 0.515152 -0.094697 4.286558 2.925762 0.252174
}
```



Вариант 2.

```
Dragon {
0.824074 0.281482 -0.212346 0.864198 -1.882290 -0.110607 0.787473
0.088272 0.520988 -0.463889 -0.377778 0.785360 8.095795 0.212527
}
```



Вариант 3.

```
Curve_Koch_3 {
0.307692 0.000000 0.000000 0.294118 4.119164 1.604278 0.151515
0.192308 -0.205882 0.653846 0.088235 -0.688840 5.978916 0.253788
0.192308 0.205882 -0.653846 0.088235 0.668580 5.962514 0.253788
0.307692 0.000000 0.000000 0.294118 -4.136530 1.604278 0.151515
0.384615 0.000000 0.000000 -0.294118 -0.007718 2.941176 0.189394
}
```



Вариант 4.

```
Swirl {
0.745455 -0.459091 0.406061 0.887121 1.460279 0.691072 0.912675
-0.424242 -0.065152 -0.175758 -0.218182 3.809567 6.741476 0.087325
}
```



Вариант 5.

```
Coral {
0.307692 -0.531469 -0.461538 -0.293706 5.401953 8.655175 0.40
0.307692 -0.076923 0.153846 -0.447552 -1.295248 4.152990 0.15
0.000000 0.545455 0.692308 -0.195804 -4.893637 7.269794 0.45
}
```



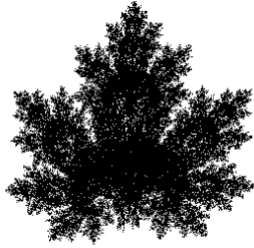
Вариант 6.

```
Leaf {
```

```

0.14  0.01  0.00  0.51 -0.08 -1.31  0.06
0.43  0.52 -0.45  0.50  1.49 -0.75  0.37
0.45 -0.49  0.47  0.47 -1.62 -0.74  0.36
0.49  0.00  0.00  0.51  0.02  1.62  0.21
}

```

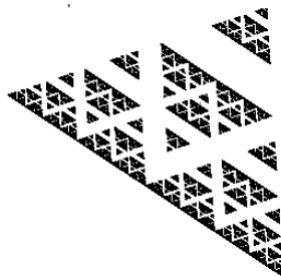


Вариант 7.

```

Triangle_2 {
-0.40  0.00  0.00 -0.40  0.24  0.37  0.23
 0.50  0.00  0.00  0.50 -1.37  0.25  0.36
 0.21  0.00  0.00  0.21  1.00  1.47  0.06
 0.50  0.00  0.00  0.50  0.76 -1.16  0.36
}

```



Вариант 8.

```

Dendrit_1 {
0.5  0.5  0.5 -0.5  0.0  0.0  0.5
0.6 -0.2 -0.2 -0.6  0.4  0.2  0.5
}

```

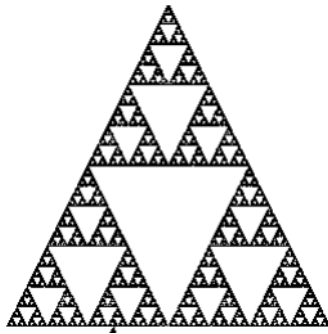


Вариант 9.

```

Sierpinsky {
0.5  0.0  0.0  0.5  0.00  0.5  0.333333
0.5  0.0  0.0  0.5 -0.25  0.0  0.333333
0.5  0.0  0.0  0.5  0.25  0.0  0.333333
}

```



Вариант 10.

```
Fir_1 {
0.0 -0.7 0.7 0.0 0.0 0.0 0.5
0.7 0.0 0.0 0.7 0.3 0.0 0.5
}
```



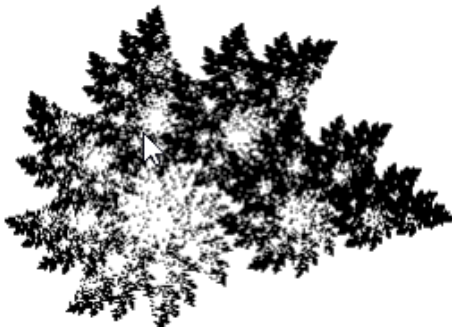
Вариант 11.

```
Curve_Koch {
0.5 0.288675 0.288675 -0.5 0.0 0.000000 0.5
0.5 -0.288675 -0.288675 -0.5 0.5 0.288675 0.5
}
```



Вариант 12.

```
Leaf_3 {
0.60 -0.6 0.6 0.60 0.00 0.0 0.5
0.53 0.0 0.0 0.53 0.47 0.0 0.5
}
```



Вариант 13.

```
Wolfs {
0.5 0.5 -0.5 0.5 -1.0 0.0 0.5
```

```
0.5 0.5 0.5 -0.5 0.0 1.0 0.5
}
```



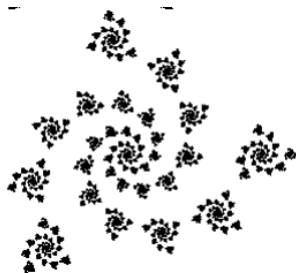
Вариант 14.

```
Fir_2 {
  0.1000 0.0000 0.0000 0.1600 0.0 0.0 0.01
  0.8500 0.0000 0.0000 0.8500 0.0 1.6 0.85
 -0.1667 -0.2887 0.2887 -0.1667 0.0 1.6 0.07
 -0.1667 0.2887 -0.2887 -0.1667 0.0 1.6 0.07
}
```



Вариант 15.

```
Spiral_12 {
 -0.61 0.7 -0.7 -0.61 0.00 0.0 0.9
 0.21 0.0 0.0 0.21 0.79 0.0 0.1
}
```



Вариант 16.

```
Talon {
 0.7 0.2 -0.2 0.7 0.0 0.0 0.5
 0.0 -0.6 0.6 0.0 1.0 -0.6 0.5
}
```



Вариант 17.

```
Spiral_19 {
0.2 -0.3 0.3 0.2 0.0 0.0 0.3
0.5 0.7 -0.7 0.5 0.5 0.7 0.7
}
```



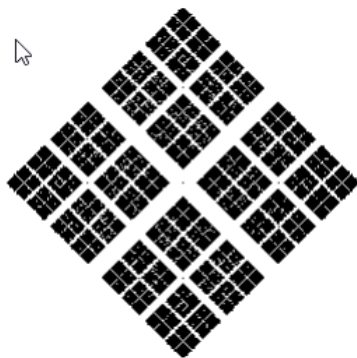
Вариант 18.

```
Chain_2 {
-0.75 -0.30 0.30 -0.75 0.0 0.00 0.7
0.30 -0.35 0.35 0.30 0.7 -0.35 0.3
}
```



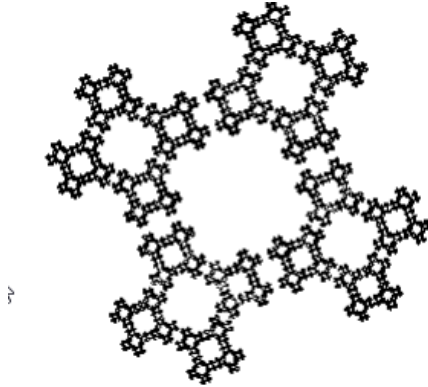
Вариант 19.

```
Square {
0.00 1.0 -1.0 0.00 0.00 0.0 0.9
0.46 0.0 0.0 0.46 0.54 0.0 0.1
}
```



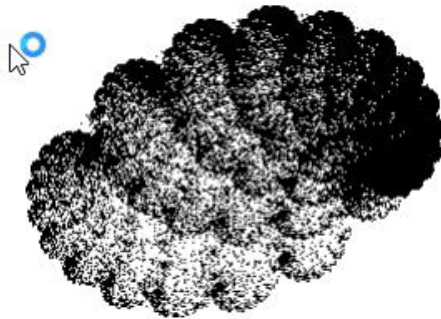
Вариант 20.

```
Lattice {  
0.3 -0.3 0.3 0.3 0.0 0.0 0.13  
0.0 1.0 -1.0 0.0 1.0 1.0 0.87  
}
```



Вариант 21.

```
Seashell {  
-0.92 -0.09 0.09 -0.92 0.00 0.00 0.1  
0.79 0.26 -0.26 0.79 0.21 0.26 0.9  
}
```



Вариант 22.

```
Parachute {  
0.336735 0.000000 0.000000 0.330097 -0.161633 0.610194 0.143  
0.326531 0.000000 0.000000 0.330097 0.143265 0.610194 0.143  
0.091837 -0.281553 0.336735 0.106796 -0.247188 0.551960 0.143  
0.275510 -0.174757 0.214286 0.271845 -0.061759 0.370832 0.143  
0.265306 0.184466 -0.204082 0.262136 0.041585 0.372231 0.143  
0.102041 0.300971 -0.336735 0.097087 0.222962 0.545807 0.143  
0.326531 0.000000 0.010204 0.339806 -0.006735 0.104714 0.143  
}
```

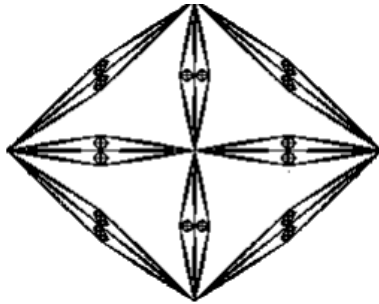


Вариант 23.

```

Diamond {
0.5 0.0 0.4 0.1 1.00 1.20 0.125
0.5 0.0 0.0 0.1 1.00 2.00 0.125
0.5 0.0 -0.4 0.1 1.00 2.80 0.125
0.5 0.0 0.0 0.1 -1.00 2.00 0.125
0.5 0.0 0.4 0.1 -1.00 2.80 0.125
0.5 0.0 -0.4 0.1 -1.00 1.20 0.125
0.0 0.1 0.4 0.0 -0.22 1.42 0.125
0.0 0.1 0.4 0.0 -0.22 3.02 0.125
}

```

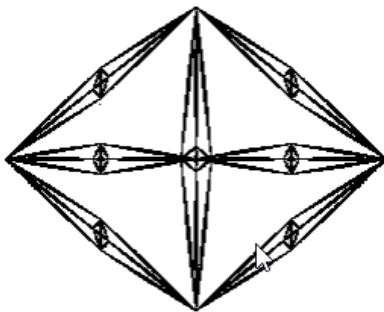


Вариант 24.

```

Diamond_2 {
0.5 0.0 0.4 0.1 1.00 1.20 0.14
0.5 0.0 0.0 0.1 1.00 2.00 0.14
0.5 0.0 -0.4 0.1 1.00 2.80 0.14
0.5 0.0 0.0 0.1 -1.00 2.00 0.14
0.5 0.0 0.4 0.1 -1.00 2.80 0.14
0.5 0.0 -0.4 0.1 -1.00 1.20 0.14
0.0 0.1 0.8 0.0 -0.22 2.22 0.14
}

```

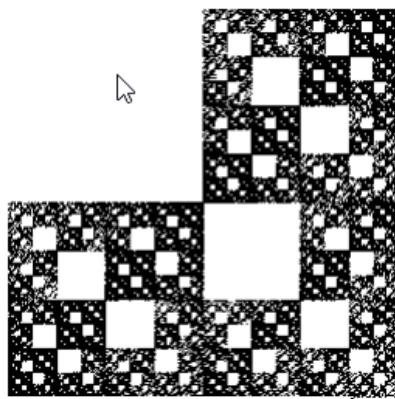


Вариант 25.

```

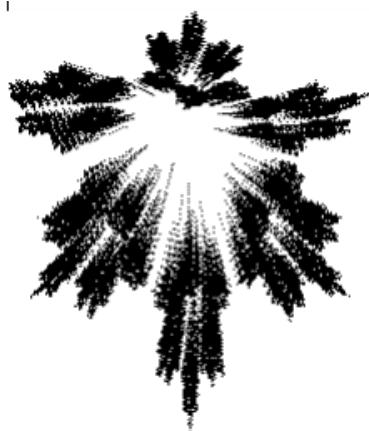
Sierpinsky_3 {
0.333333 0 0 0.333333 -0.333333 0.733333 0.125
0.333333 0 0 0.333333 -0.033333 0.733333 0.125
0.333333 0 0 0.333333 0.266667 0.733333 0.125
0.333333 0 0 0.333333 -0.333333 0.433333 0.125
0.333333 0 0 0.333333 0.266667 0.433333 0.125
0.333333 0 0 0.333333 -0.333333 0.133333 0.125
0.333333 0 0 0.333333 -0.033333 0.133333 0.125
0.333333 0 0 0.333333 0.266667 0.133333 0.125
}

```



Вариант 26.

```
Shooter {
-0.322222 -0.007143  0.040741 -0.240476  0.164143 10.217592 0.053858
-0.166667 -0.250000 -0.333333  0.214286  3.914220  3.198747 0.082436
-0.166667  0.250000  0.333333  0.214286 -3.819525  3.171691 0.082436
 0.111111 -0.285714 -0.166667 -0.107143  4.455358  8.163677 0.041218
 0.155556 -0.257143 -0.207407 -0.133333 -1.593255  8.570432 0.051293
-0.222222  0.000000  0.000000  0.428571  0.049603 -0.129243 0.065949
 0.944444 -0.007143 -0.007407  0.952381  0.038005  0.363189 0.622811
}
```



Вариант 27.

```
Harter-Heighway dragon {
-0.75 -0.30 0.30 -0.75 0.00 0.00 0.7
0.30 -0.35 0.35 0.30 0.70 -0.35 0.3;
}
```

